Emergency Recovery
Replication Slot Deactivated During Active Restore

Item	Detail
PostgreSQL Version	16
Replication Slot Name	ssncpri
Data Directory	/apps/pgsql_data/16
Current Status	Restore running on Primary; slot deactivated; standby way behind

SITUATION: Data restore is actively running on the Primary and must NOT be interrupted. The replication slot `ssncpri` has been deactivated and the standby is no longer receiving WAL changes. The standby is significantly behind the primary.

There are two possible outcomes depending on whether the WAL segments the standby needs are still on disk. You must diagnose first, then follow the appropriate recovery path.

Scenario	WAL Still on Disk?	Recovery Path
Scenario A (Best Case)	Yes — WAL files exist	Re-enable slot, reconnect standby, no rebuild needed
Scenario B (Worst Case)	No — WAL already removed	Let restore finish, then full standby rebuild with pg_basebackup

STEP 1: Diagnose the Situation on PRIMARY

> **DO NOT stop the restore.** All diagnostic steps run on the PRIMARY without interrupting the active restore.

1.1 — Check the Replication Slot Status

SQL — Check Slot ssncpri

```
-- Run on PRIMARY:
SELECT slot_name,
       slot_type,
       active,
       restart_lsn,
       confirmed_flush_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
 slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
```

What to look for:

- **active = f** — Confirms the slot is deactivated
- **slot_lag** — Shows how far behind the standby is (e.g. 30 GB, 80 GB, etc.)
- **restart_lsn** — The LSN position the standby needs WAL from

1.2 — Check if the Required WAL Segments Still Exist

SQL — Find Required WAL File Name

```
-- Run on PRIMARY:
-- Get the WAL file name the standby needs
SELECT pg_walfile_name(restart_lsn) AS needed_wal_file
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
```

SQL — Check if WAL File Exists in pg_wal Directory

```
-- Now check if that WAL file still exists on disk:
-- Replace the filename from the above query result
SELECT name, pg_size_pretty(size)
FROM pg_ls_waldir()
WHERE name >= '<needed_wal_file_from_above>'
ORDER BY name
LIMIT 5;
```

BASH — Direct File Check

```
# Alternative: check directly from bash
ls -la /apps/pgsql_data/16/pg_wal/ | grep '<needed_wal_file_from_above>'
```

1.3 — Check Overall WAL Directory Size and Disk Space

SQL — WAL Directory Size

```
-- WAL directory total size:
SELECT pg_size_pretty(sum(size)) AS total_wal_on_disk
FROM pg_ls_waldir();
```

BASH — Disk Space Check

```
# Disk space available:
df -h /apps/pgsql_data/16/pg_wal
```

1.4 — Check if the Standby is Still Trying to Connect

SQL — Check Active Replication Connections

```
-- Run on PRIMARY:
SELECT pid, usename, client_addr, state, sent_lsn, replay_lsn
FROM pg_stat_replication;
-- If empty = standby is NOT connected at all
-- If present but state != 'streaming' = standby is stuck
```

1.5 — Check Standby Logs for Errors

BASH — Check Standby Logs

```
# Run on STANDBY:
tail -100 /apps/pgsql_data/16/log/postgresql-*.log | grep -i 'error\|fatal\|
wal'
```

Look for these messages:

- `requested WAL segment has already been removed` — WAL is gone, Scenario B
- `could not receive data from WAL stream` — Connection lost but WAL may still exist
- `replication slot "ssncpri" does not exist` — Slot was dropped

STEP 2: Decision Point

Based on Step 1 results, follow ONE of the two paths below:

- If the needed WAL file EXISTS on disk → Go to **STEP 3: Scenario A** (Reconnect without rebuild)
- If the needed WAL file is GONE → Go to **STEP 4: Scenario B** (Rebuild after restore completes)

STEP 3: Scenario A — WAL Still Exists (Reconnect Without Rebuild)

GOOD NEWS: The WAL segments the standby needs are still on disk. You can recover without rebuilding. The restore continues uninterrupted.

3.1 — Immediately Protect WAL from Being Removed

First, prevent PostgreSQL from recycling the WAL segments you need. Run on PRIMARY:

SQL — Protect WAL on Primary (No Restart Needed)

```
-- Increase wal_keep_size to protect existing WAL (takes effect immediately
with reload)
ALTER SYSTEM SET wal_keep_size = '100GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
SELECT pg_reload_conf();

-- Verify the change took effect:
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
```

This is time-critical. Every checkpoint that runs on the primary could remove the WAL segments you need. Run these commands IMMEDIATELY.

3.2 — Check if the Slot Needs to Be Recreated or Just Reactivated

SQL — Re-check Slot

```
-- Check slot status again:
SELECT slot_name, active, restart_lsn
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
```

If the slot still exists (active = f):

- The slot itself is fine — it just needs the standby to reconnect to it
- Proceed to Step 3.3

If the slot does NOT exist (no rows returned):

SQL — Recreate Slot if Missing

```
-- Recreate the slot (this reserves WAL from current position forward)
SELECT pg_create_physical_replication_slot('ssncpri', true);

-- Verify:
SELECT slot_name, active, restart_lsn FROM pg_replication_slots;
```

3.3 — Restart the Standby to Force Reconnection

BASH — Restart Standby

```
# Run on STANDBY:
sudo systemctl restart postgresql-16

# Or using pg_ctl:
pg_ctl -D /apps/pgsql_data/16 restart
```

3.4 — Verify Standby Reconnected and Is Streaming

SQL — Verify Standby Is Streaming

```
-- On STANDBY: verify it's in recovery and streaming
SELECT pg_is_in_recovery();             -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

SQL — Verify Slot Active and Lag Decreasing

```
-- On PRIMARY: verify slot is active and lag is decreasing
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
 slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
-- active must be: t
-- slot_lag should start decreasing over time
```

SQL — Monitor Lag Recovery

```
-- Run this every few minutes to confirm lag is shrinking:
SELECT client_addr, state,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn)) AS se
nd_lag
FROM pg_stat_replication;
-- state must be: streaming
-- replay_lag should decrease steadily
```

If the standby is streaming and lag is decreasing, recovery is successful. The standby will catch up on its own. Continue monitoring until the restore completes and lag drops to near zero.

3.5 — After Restore Completes, Clean Up

SQL — Revert WAL Protection After Recovery

```
-- After restore is finished and standby has caught up:
-- On PRIMARY: revert the wal_keep_size back to normal
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;
SELECT pg_reload_conf();

-- Verify:
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
```

STEP 4: Scenario B — WAL Already Removed (Full Standby Rebuild)

WAL segments are gone. The standby cannot catch up via streaming. You must let the restore finish on the primary, then rebuild the standby from scratch using pg_basebackup.

4.1 — While Restore Is Running: Prevent Further Damage

Even though recovery requires a rebuild, protect WAL going forward so the new standby won't face the same problem:

SQL — Protect WAL for Future Standby

```
-- On PRIMARY: set WAL protection now (no restart needed)
ALTER SYSTEM SET wal_keep_size = '100GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
SELECT pg_reload_conf();
```

4.2 — While Restore Is Running: Stop the Standby

The standby is wasting resources retrying connections that will never succeed. Stop it cleanly:

BASH — Stop Standby

```
# On STANDBY: stop PostgreSQL
sudo systemctl stop postgresql-16
```

4.3 — While Restore Is Running: Drop the Broken Slot

The broken slot is holding WAL references that are useless now. Drop it to free resources:

SQL — Drop Broken Replication Slot ssncpri

```
-- On PRIMARY: drop the broken slot
SELECT pg_drop_replication_slot('ssncpri');

-- Verify it's gone:
SELECT slot_name, active FROM pg_replication_slots;
```

Note: Do NOT create a new slot yet. Wait until the restore is complete and you are ready to run pg_basebackup. The -C flag in pg_basebackup will create the new slot automatically.

4.4 — Wait for Restore to Complete

Monitor the restore progress on the primary. Do not proceed to the rebuild until the restore (including FK creation, indexes, and all post-load steps) is fully complete.

SQL — Monitor Restore Progress

```sql
-- On PRIMARY: monitor active restore queries
SELECT pid, now() - xact_start AS duration, state, query
FROM pg_stat_activity
WHERE state = 'active'
  AND query NOT LIKE '%pg_stat_activity%'
ORDER BY duration DESC;
```

4.5 — After Restore Completes: Rebuild the Standby

The restore is now complete. Time to rebuild the standby from scratch.

Step A — Wipe the standby data directory:

BASH — Wipe Standby Data Directory

```
# On STANDBY: make absolutely sure PostgreSQL is stopped
sudo systemctl stop postgresql-16

# Verify no processes are running
ps aux | grep postgres

# CRITICAL: Triple-check you are on the STANDBY server, NOT the primary!
hostname

# Remove the old data directory contents
rm -rf /apps/pgsql_data/16/*
```

DANGER: The rm -rf command above will DESTROY all data. Verify you are on the STANDBY server before running.

Step B — Run pg_basebackup from the primary:

BASH — pg_basebackup with Slot Creation

```
# On STANDBY: run pg_basebackup
pg_basebackup \
  -h <primary_host_or_ip> \
  -p 5432 \
  -U replication_user \
  -D /apps/pgsql_data/16 \
  -Fp \
  -Xs \
  -P \
  -R \
  -S ssncpri \
  -C
```

Flags explained:

- **-h <primary_host_or_ip>** — Replace with your actual primary hostname or IP
- **-p 5432** — PostgreSQL port (change if different)
- **-U replication_user** — Replace with your actual replication user
- **-D /apps/pgsql_data/16** — Target data directory on standby
- **-Fp** — Plain format (direct file copy)

- **-Xs** — Stream WAL during backup (ensures consistency)
- **-P** — Show progress percentage
- **-R** — Auto-creates `standby.signal` and configures `primary_conninfo` in `postgresql.auto.conf`
- **-S ssncpri** — Use this replication slot name
- **-C** — Create the replication slot on the primary (since we dropped it in Step 4.3)

Step C — Fix ownership and start the standby:

BASH — Fix Permissions and Start Standby

```
# On STANDBY: fix file ownership (if pg_basebackup was run as root)
chown -R postgres:postgres /apps/pgsql_data/16

# Start the standby
sudo systemctl start postgresql-16
```

4.6 — Verify the New Standby Is Working

SQL — Verify Standby Is In Recovery and Streaming

```
-- On STANDBY:
SELECT pg_is_in_recovery();             -- Must return: t
SELECT status FROM pg_stat_wal_receiver; -- Must return: streaming
```

SQL — Verify Slot ssncpri Is Active

```
-- On PRIMARY: verify slot is active and replication is flowing
SELECT slot_name, active,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS
 slot_lag
FROM pg_replication_slots
WHERE slot_name = 'ssncpri';
-- active must be: t
-- slot_lag should be very small (near 0)
```

SQL — Full Replication Health Check

```
-- On PRIMARY: verify full replication health
SELECT pid, client_addr, state, sent_lsn, replay_lsn,
       pg_size_pretty(pg_wal_lsn_diff(sent_lsn, replay_lsn)) AS replay_lag
FROM pg_stat_replication;
-- state must be: streaming
-- replay_lag should be minimal
```

If the standby shows streaming with minimal lag, the rebuild is complete.

4.7 — Clean Up: Revert WAL Protection Settings

SQL — Revert WAL Protection

```
-- On PRIMARY: revert the temporary WAL protection
ALTER SYSTEM RESET wal_keep_size;
ALTER SYSTEM RESET max_slot_wal_keep_size;
SELECT pg_reload_conf();

-- Verify:
SHOW wal_keep_size;
SHOW max_slot_wal_keep_size;
```

Quick Decision Reference

Step	Action	Where to Run
1	Check slot status: active = f?	PRIMARY
2	Find needed WAL file name from restart_lsn	PRIMARY
3	Check if that WAL file exists in pg_wal	PRIMARY
4a	WAL EXISTS: Set wal_keep_size, reload, restart standby	PRIMARY + STANDBY
4b	WAL GONE: Stop standby, drop slot, wait for restore to finish	PRIMARY + STANDBY
5b	After restore done: wipe standby, pg_basebackup -R -S ssncpri -C	STANDBY
6	Verify streaming replication is healthy	PRIMARY + STANDBY
7	Revert wal_keep_size and max_slot_wal_keep_size	PRIMARY

GOLDEN RULE: Never stop or interrupt the active restore on the primary. Replication can always be rebuilt — lost restore progress cannot.

PREVENTION FOR NEXT TIME: Before starting any large restore, always set these on the primary:

```
ALTER SYSTEM SET wal_keep_size = '100GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
SELECT pg_reload_conf();
```

These two commands protect the replication slot from being invalidated during heavy WAL generation and cost you nothing except disk space for retained WAL.